

The list comprehension takes place inside the square brackets (`[]`). We use `for word in word_list` to iterate over every word in our `word_list`. For each word, we call `re.finditer()`, which finds the selected word in our text and returns a list of every location where that word occurs. We iterate over these locations, and each individual location is stored in `m`. When we access `m.start()` and `m.end()`, we'll get the location in the text of the beginning and end of the word, respectively. Notice—and get used to—the order of the `for` loops, since some people find it the opposite of the order they expected.

The whole list comprehension is enveloped by `list(set())`. This is a convenient way to get a list that contains only unique values with no duplicates. Our list comprehension alone might have multiple identical elements, but converting it to a set automatically removes duplicates, and then converting it back to a list puts it in the format we want: a list of unique word locations. You can run `print(locs)` to see the result of the whole operation:

---

```
[(17, 23), (7, 16), (0, 3), (35, 38), (4, 7)]
```

---

In Python, ordered pairs like these are called *tuples*, and these tuples show the locations of each word from `word_list` in our text. For example, when we run `text[17:23]` (using the numbers from the third tuple in the preceding list), we find that it's `divine`. Here, `d` is the 17th character of our text, `i` is the 18th character of our text, and so on until `e`, the final letter of `divine`, is the 22nd character of our text, so the tuple is rounded off with 23. You can check that the other tuples also refer to the locations of words in our `word_list`.

Notice that `text[4:7]` is `one`, and `text[7:16]` is `perfectly`. The end of the word `one` runs into the beginning of the word `perfectly` without any intervening space. If we hadn't noticed that immediately by reading the text, we could have caught it by looking at the tuples `(4, 7)` and `(7, 16)` in our `locs` variable: since 7 is the second element of `(4, 7)` and also the first element of `(7, 16)`, we know that one word ends in the same index where another word begins. In order to find places where we need to insert spaces, we'll look for cases like this: where the end of one valid word is at the same place as the beginning of another valid word.